

データ構造 区間クエリ Disjoint Sparse Table Sqrt Tree 逆アッカーマン関数 モノイド

静的な列の区間積クエリ

1. 概要

本講座では Sqrt Tree などに続き、再び静的なモノイド列に対する区間積クエリの問題を扱います。

特に、長さ N の列について $O(N)$ 時間の事前計算を前提とした場合、クエリあたり $O(\alpha(N))$ 時間 ($\alpha(N)$ は逆アッカーマン関数) での計算が可能で、このことの解説が本講座の主目標です。アイデアとしては、Sqrt Tree の単純な一般化にあたるので、Sqrt Tree について理解していれば本講座の内容も理解しやすいと思います。

競技プログラミングにおいて $\alpha(N)$ が計算量解析に登場する問題としては、他に Union-Find が有名だと思います。本講座での計算量解析はおそらく Union-Find よりも易しいので、計算量解析に $\alpha(N)$ が現れる事例を学んでみるという目的にも適していると思います。

2. 前提知識

DST (Disjoint Sparse Table) および Sqrt Tree の理解を仮定します。

アッカーマン関数、逆アッカーマン関数 を参照する箇所がありますが、本講座の議論を追う上でこの講座全体を理解している必要はありません。

3. 問題設定

本講座で扱う問題は、Disjoint Sparse Table **【問題 1】** と同一です。再掲します。

【問題 1】

$(M, *)$ をモノイドとします。 M の元からなる長さ N の列 $A = (A_0, A_1, \dots, A_{N-1})$ が与えられます。 Q 個のクエリに答えてください。 クエリでは $0 \leq L < R \leq N$ を満たす整数 L, R が与えられるので、

$$A_L * A_{L+1} * \dots * A_{R-1}$$

を出力してください。

また、計算量解析に際しては、 M の元に対する操作（配列への読み書きやモノイド演算）が $O(1)$ 時間であるとします。

4. クエリあたり k 回のモノイド演算で解く方法

4.1. $k = 0, 1, 2$ の場合

DST, Sqrt Tree はいずれも適切な事前計算のもと、 $O(1)$ 時間で区間積クエリに答えるものです。 しかし同じ「 $O(1)$ 時間」といっても少し違いがあります。 DST では、区間積クエリに対して 1 回のモノイド演算で答を求めます。 Sqrt Tree では、区間積クエリに対して 2 回のモノイド演算で答を求めます。

そこで 4 節では k を定数として、「適切な事前計算のもと、区間積クエリに k 回のモノイド演算で答を求める」という問題を考えます。 $k = 1$ の場合が DST、 $k = 2$ の場合が Sqrt Tree ということになります。

またこの問題には $k = 0$ という自明な場合があります。 すべての区間に対する答えを事前計算し、格納しておくことで区間積クエリにモノイド演算 0 回で答えられます。

これらの解法の関係は以下ようになります。

解法	事前計算	クエリ処理
全区間に対する事前計算	$O(N^2)$	$O(1)$ （モノイド演算 0 回）
DST	$O(N \log N)$	$O(1)$ （モノイド演算 1 回）
Sqrt Tree	$O(N \log \log N)$	$O(1)$ （モノイド演算 2 回）

4.2. 方針

Sqrt Tree は、 $k = 0$ の場合のデータ構造を利用して、 $k = 2$ の場合のデータ構造を作ったものと見なすことができます。この手法を復習し、一般化しましょう。

k を非負整数とします。

A を長さ N のモノイド列とします。適当なブロックサイズ S をとって、 A を $\lceil N/S \rceil$ 個のブロックに分割します。ここで以下のものを事前計算します。

- Prefix : 各 A_i について、ブロックの先頭から A_i までの総積を計算する。
- Suffix : 各 A_i について、 A_i からブロックの末尾までの総積を計算する。
- Between : 各ブロックの総積を並べた列 $(b_0, b_1, \dots)$ に対して **何らかの事前計算** を行うことで、 b に対する区間積クエリに k 回のモノイド演算で答えられるようにする。

というようにすれば、区間積クエリ $[L, R)$ のうち A_L, A_{R-1} が同一ブロックに含まれる場合 **以外**に、 $k + 2$ 回のモノイド演算で答えられるようになります。

各ブロック内で再帰的に同様のブロック分割を行っていくことで、Sqrt Tree と同様の木構造が得られ、各区間積クエリに対して $k + 2$ 回のモノイド演算で答えられるようになります。

S を適切にとることで、各深さで必要な事前計算の計算量および木の高さ H を抑えます。この部分は 4.3 節で詳しく扱います。

この過程において、Between で「区間積クエリに k 回のモノイド演算で答えられる」ための事前計算が登場しています。この問題は、本問題が k について帰納的に解けることを意味しています。

4.3. 計算量解析

本節において単に**計算量**といえば、モノイド演算の回数を表すこととします。

次の α_k は、アッカーマン関数、逆アッカーマン関数 **【定義 8】** において g_k として導入したものに等しいです。

【定義 2】

非負整数 k と正整数 N に対して、 $\alpha_k(N)$ を次の式によって定義する。

$$\begin{aligned}\alpha_0(N) &= \left\lceil \frac{N}{2} \right\rceil, \\ \alpha_k(1) &= 0, \\ \alpha_{k+1}(N) &= 1 + \alpha_{k+1}(\alpha_k(N)) \quad (k \geq 0, N \geq 2).\end{aligned}$$

定義式は、次のように書くこともできます。

$$\alpha_{k+1}(N) = \min \left\{ n \mid \alpha_k^{(n)}(N) = 1 \right\}.$$

つまり $\alpha_{k+1}(N)$ は、 N に対して α_k をとる操作を反復したときに 1 に到達するまでの操作回数です。

以上の準備のもと、区間積クエリに k 回のモノイド演算で答えられるようにするための事前計算の計算量を解析します。 k が奇数の場合、偶数の場合についてほとんど同じ議論が適用可能ですが、本講座では簡単のため、 k が奇数の場合のみ扱います。

【命題 3】

k を正整数とする。このとき、任意の正整数 N に対して計算量 $(2k - 1)N \cdot \alpha_k(N)$ の事前計算を行うことで、区間積クエリに $(2k - 1)$ 回のモノイド演算で答えられるようにできる。

k に関する帰納法で証明します。 $k = 1$ の場合は $\alpha_1(N) = \lceil \log_2 N \rceil$ より DST が条件を満たします。

k を正整数とし、 k の場合の成立を仮定します。

長さ N の列に対してブロックサイズ S を $S = \alpha_k(N)$ ととり、4.2 節で述べた方針での事前計算をします。ブロックの総積の列について、 k 回のモノイド演算で区間積クエリに答えられるような事前計算を行い、木を区間の長さが 1 になるまで再帰的に構築しましょう。

木の高さ H は、 N に対して α_k を反復して 1 に到達するまでにかかる回数です。したがって $H = \alpha_{k+1}(N)$ となります。

深さ 0 で行われる事前計算の計算量はどのようなのでしょうか？

- Prefix, Suffix：それぞれ N 以下。

- Between：帰納法の仮定より、 $M = \lfloor N/\alpha_k(N) \rfloor$ とするとき、 $(2k-1)M \cdot \alpha_k(M)$ の計算量.

▶ $M = \lfloor N/\alpha_k(N) \rfloor$ について

$\alpha_k(M) \leq \alpha_k(N)$ より

$$(2k-1)M \cdot \alpha_k(M) \leq (2k-1) \cdot \frac{N}{\alpha_k(N)} \cdot \alpha_k(N) = (2k-1)N$$

となるので、深さ 0 で行われる事前計算量は $(2k+1)N$ 以下になります。同様に考えると、深さを固定した場合の事前計算量は $(2k+1)N$ 以下、全体では $(2k+1)NH = (2k+1)N \cdot \alpha_{k+1}(N)$ となります。

よって帰納法により **【命題 3】** が示されました。 ■

4.4. 結論

$k = 0, 1, 2, \dots$ に対して、以下の表の計算量の事前計算のもと、区間積クエリに k 回のモノイド演算で答えることができます。

クエリあたりのモノイド演算 k	事前計算の計算量
$k = 0$	$O(N^2)$
$k = 1$	$O(N\alpha_1(N)) = O(N \log N)$
$k = 2$	$O(N \log \log N)$
$k = 3$	$O(N\alpha_2(N)) = O(N \log^* N)$
$k = 4$	$O(N\alpha_2(N)) = O(N \log^* N)$
$k = 5$	$O(N\alpha_3(N))$
$k = 6$	$O(N\alpha_3(N))$
$\vdots$	$\vdots$

k が奇数の場合は 4.3 で示した通りです。 k が偶数の場合も同様に証明できます。

5. 逆アッカーマン関数による計算量解析

4.4 節で解説したように、クエリあたりのモノイド演算の回数上限を増やすほど、事前計算の計算量は小さくなるというトレードオフがあります。 $N = Q$ 程度の状況でこれらのバランスをとると、計算量に逆アッカーマン関数が登場します。

アッカーマン関数、逆アッカーマン関数 **【定義 10】** を再掲します。

【定義 4】

正整数 N に対して、 $\alpha(N)$ を次の式によって定義する。

$$\alpha(N) = \min\{k \mid \alpha_k(N) \leq 3\}.$$

α を**逆アッカーマン関数** (inverse Ackermann function) という。

【定理 5】

計算量 $O(N\alpha(N))$ の事前計算のもと、区間積クエリに $O(\alpha(N))$ 時間で答えることができる。したがって **【問題 1】** を $O((N + Q)\alpha(N))$ 時間で解くことができる。

【命題 3】 を $k = \alpha(N)$ として用います。 $\alpha_k(N) \leq 3$ なので、計算量 $(2\alpha(N) - 1)N \cdot 3$ の事前計算を行うことで、区間積クエリに $2\alpha(N) - 1$ 回のモノイド演算で答えられます。

区間積クエリの計算量をモノイド演算の回数と比例するものとするためには、事前計算された要素のうち必要なものにひとつあたり $O(1)$ 時間でアクセスすることが必要となりますが、その方法は **Sqrt Tree** で解説したものと同様なので、ここでは省略します。

これで **【定理 5】** の計算量が達成できています。 ■

さらに少しの工夫で事前計算の計算量を $O(N)$ 時間とすることもできます。

【定理 6】

$O(N)$ 時間の事前計算のもと、区間積クエリに $O(\alpha(N))$ 時間で答えることができる。したがって **【問題 1】** を $O(N + Q\alpha(N))$ 時間で解くことができる。

ここでも 4 節で解説したようなブロック分割の手法を考えます。ブロックサイズ S を $S = \alpha(N)$ ととり、 $O(N + (N/S)\alpha(N/S))$ 時間の事前計算を行います。すると **【定理 5】** よ

り、クエリ区間がひとつのブロックに含まれない場合にはクエリに $O(\alpha(N/S))$ 時間で答えられます。

クエリ区間がひとつのブロックに含まれる場合にクエリに $O(S)$ 時間で答えられることは明らかです。 $S = \alpha(N)$ および $\alpha(N/S) \leq \alpha(N)$ より、クエリに $O(\alpha(N))$ 時間で答えられることが分かりました。

$$(N/S)\alpha(N/S) \leq (N/S)\alpha(N) = N$$

なので、事前計算は $O(N)$ 時間になっています。 ■

6. 計算量下界について

区間積クエリに **オンライン**で答えることを前提とします。オンラインとは、クエリをまとめて事前に知ることができない（言い換えれば、クエリに答えるたびに次のクエリが与えられる）ということです。

この場合、4.4 節および 5 節で述べた計算量オーダーは、適切な計算モデルのもとで理論上最適であることが知られています。詳しくは [文献 1](#) を参考にしてください。

7. 関連問題

1. <https://qoj.ac/contest/1356/problem/7182>

この問題は [Sqrt Tree](#) でも関連問題として挙げた問題です。解法として Sqrt Tree が想定された問題ですが、解説では本講座に相当する内容も解説されています。

8. まとめ

AtCoder Algorithm Lectures では、これまで [【問題 1】](#) を [DST \(Disjoint Sparse Table\)](#)、[Sqrt Tree](#) でも扱ってきました。本講座はそれらの集大成にあたる講座で、ある意味では理論上最適な計算量を達成する方法を解説できました。競技プログラミングの問題を解く上ではほとんど役に立たない知識だと思いますが、楽しんでいただければ幸いです。

なお、区間最小値クエリなどの状況では、[【問題 1】](#) は $O(N + Q)$ 時間で解けることが知られています（**線形時間 RMQ**）。これは本講座の結果と矛盾するものではなく、一般のモノイ

ドにはない $\min$ 特有の性質が使われることになります。興味があればそちらも合わせて学習してみてください。

9. 参考文献

1. Noga Alon, Baruch Schieber. Optimal preprocessing for answering on-line product queries. arXiv:2406.06321. 2024.
<https://arxiv.org/abs/2406.06321>.
2. Raimund Seidel. Understanding the Inverse Ackermann Function. EuroCG 2006 invited lecture slides. 2006.
<https://cgi.di.uoa.gr/~ewcg06/invited/Seidel.pdf>. (閲覧日: 2026-03-02).
3. convexineq. 半群の区間積クエリについて. Qiita. 2021-12-22.
<https://qiita.com/convexineq/items/4b6920af3e3d37a96540>. (閲覧日: 2026-03-02).
4. sotanishy. Sqrt Treeで爆速区間クエリ. Qiita. 2021-01-10.
<https://qiita.com/sotanishy/items/89f4dd452bcd332f24>. (閲覧日: 2026-03-02).
5. Wikipedia. Range query (computer science).
[https://en.wikipedia.org/wiki/Range_query_\(computer_science\)](https://en.wikipedia.org/wiki/Range_query_(computer_science)). (閲覧日: 2026-03-02).
6. return20071007. 一种简单的 $O(n\alpha(n))$ 静态区间半群信息查询离线算法. UOJ Blog. <https://return20071007.blog.uoj.ac/blog/9292>. (閲覧日: 2026-04-15).
7. return20071007. 一种简单的 $O(n) - O(\alpha(n))$ 静态区间半群信息查询在线算法. UOJ Blog. <https://return20071007.blog.uoj.ac/blog/9298>. (閲覧日: 2026-04-15).

トップページ : [AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など : [Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。 リアルタイムのオンラインコンテストで競い合うことや、 5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)